

Companion Document: update_beta.R

q() — Variational Update for Cox Survival Coefficients

Andrew Walther

2026-03-25

Table of contents

1	Overview	2
1.1	Model Context	2
1.2	CAVI Update Order	2
2	Mathematical Background	2
2.1	Working Response and Weights	2
2.2	Partial Working Response $z^{(-k)}$	3
2.3	Precision and Signal (Error-in-Variables Correction)	3
2.4	Point-Normal Prior and Sparsity	3
2.5	Scalar vs. Vector EBNM	3
3	Function Reference	4
3.1	compute_z_no_k	4
3.2	update_beta_k	5
3.3	update_beta_all	6
4	Test Suite	7
4.1	T1: Mathematical Identity Checks (6 tests)	7
4.2	T2: Non-Informative Prior / WLS Limit (2 tests)	7
4.3	T3: Known Signal Recovery K=1 (2 tests)	7
4.4	T4: Multi-Factor K=5 (3 tests)	8
4.5	T5: Null Factor Shrinkage (2 tests)	8
4.6	T6: Error-in-Variables (1 test)	8
4.7	T7: Numerical Stability (5 tests)	8
4.8	T8: Gauss-Seidel Behaviour (2 tests)	8
4.9	T9: V2.R Consistency (1 test)	9
5	Demos	9
5.1	Demo 1: Anatomy of update_beta_k()	9
5.2	Demo 2: Signal Recovery K=1	9
5.3	Demo 3: Null Factor Shrinkage	9
5.4	Demo 4: Error-in-Variables Effect	10
5.5	Demo 5: Multi-Factor K=5	10

6	Design Decisions and V2 Improvements	10
6.1	Gauss-Seidel vs. Jacobi Updates	10
6.2	Precision Floor	10
6.3	Separation from L and F Updates	10
7	Common Pitfalls	10
8	Related Files	11

1 Overview

code/update_beta.R implements the CAVI (Coordinate Ascent Variational Inference) update for the survival coefficients in the Supervised Bayesian Matrix Factorization model. Each factor k has a scalar coefficient β_k linking the latent factor scores L to the Cox proportional hazards survival likelihood.

1.1 Model Context

The full model is:

$$Y = LF^T + E \quad (\text{genomics observation model})$$

$$h(t) = h_0(t) \exp(L\beta) \quad (\text{Cox PH survival model})$$

where: - $\mathbf{Y}$ is the $n \times p$ genomics matrix - $\mathbf{L}$ is the $n \times K$ latent factor score matrix - $\mathbf{F}$ is the $p \times K$ factor loading matrix - β is a K -vector of survival coefficients - $\mathbf{E}$ is Gaussian noise with precision

β_k **appears exclusively in the survival likelihood.** It does not enter the genomics reconstruction term. This clean separation means the update depends only on the Cox partial likelihood signal, mediated through the working response z and the precision weights w derived from the current hazard.

1.2 CAVI Update Order

$$\beta \rightarrow L \rightarrow F \rightarrow \tau \rightarrow \dots$$

The update is performed first in each CAVI sweep, using the current posterior moments of L .

2 Mathematical Background

2.1 Working Response and Weights

The Cox PH likelihood is approximated via a quadratic (Laplace/Newton) expansion. This yields:

- $\mathbf{z}$: n -vector of working responses (current residuals in the linearised survival model)
- $\mathbf{w}$: n -vector of precision weights (curvature of the log-likelihood at current values)

2.2 Partial Working Response $z^{(-k)}$

For updating β_k , the contribution of all other factors $k' \neq k$ must be subtracted from z :

$$z_i^{(-k)} = z_i - \sum_{k' \neq k} \bar{l}_{ik'} \bar{\beta}_{k'}$$

This isolates the signal relevant to factor k alone. Critically, $z^{(-k)}$ **does not depend on** l_{ik} **or** β_k — it uses only the remaining factors. This means the same $z^{(-k)}$ can be reused for both the β_k update and the L_k update if needed.

2.3 Precision and Signal (Error-in-Variables Correction)

The EBNM pseudo-observation and noise for β_k are:

$$A_k = \sum_i w_i \mathbb{E}[l_{ik}^2] \quad (\text{precision; full posterior second moment})$$

$$B_k = \sum_i w_i z_i^{(-k)} \bar{l}_{ik} \quad (\text{signal})$$

$$x_k = \frac{B_k}{A_k}, \quad s_k = \frac{1}{\sqrt{A_k}}$$

Critical distinction: A_k uses $\text{EL2}_k = \text{Var}_q(l_k) + \text{EL}_k^2$, the full posterior second moment, **not** EL_k^2 . This is the error-in-variables correction: when the latent scores L are uncertain, using only the squared posterior mean would underestimate the effective precision and bias the estimate. Using EL2_k accounts for posterior variance in L , propagating uncertainty correctly into the update.

2.4 Point-Normal Prior and Sparsity

β_k is given a point-normal prior:

$$\beta_k \sim \pi_0 \delta(0) + (1 - \pi_0) \mathcal{N}(0, \sigma_\beta^2)$$

This prior promotes sparsity. Factors with weak survival signal are shrunk toward zero; factors with strong, consistent signal are retained with their estimated magnitude. The EBNM call handles this automatically, returning posterior mean, SD, and second moment.

2.5 Scalar vs. Vector EBNM

The update is a **scalar EBNM**: each factor k yields a single pseudo-observation x_k . This contrasts with the L and F updates, which are vector EBNM problems (one pseudo-observation per sample or gene). The scalar nature follows from β_k being a single coefficient shared across all n samples.

3 Function Reference

The file contains three functions with a clean separation of concerns:

Function	Role	Returns
<code>compute_z_no_k</code>	Partial working response for factor k	n-vector
<code>update_beta_k</code>	Single-factor EBNM update	Named list with posterior moments
<code>update_beta_all</code>	Gauss-Seidel loop over all K factors	Named list with K-vectors

3.1 `compute_z_no_k`

```
compute_z_no_k(z, EL, EBeta, k)
```

Purpose: Subtract the contributions of all factors except k from the working response, isolating the signal for β_k .

Implementation:

```
z - (EL %*% EBeta) + EL[, k] * EBeta[k]
```

This is numerically efficient: rather than summing over all $k' \neq k$ explicitly, it computes the full linear predictor `EL %*% EBeta` and then adds back the k-th term `EL[,k] * EBeta[k]`.

Parameters:

Parameter	Type	Description
<code>z</code>	n-vector	Working response from Cox likelihood
<code>EL</code>	$n \times K$ matrix	Posterior means of factor scores
<code>EBeta</code>	K-vector	Current posterior means of
<code>k</code>	integer	Factor index to exclude

Returns: n-vector $z^{(-k)}$

Key insight: Because $z^{(-k)}$ uses only $k' \neq k$ terms, it does not change when β_k or l_{ik} change. The same $z^{(-k)}$ is safe to reuse across both the β_k and `L_k` updates within a single CAVI iteration.

3.2 update_beta_k

```
update_beta_k(w, z_no_k, EL_k, EL2_k, prior_family = "point_normal", A_floor = 1e-10)
```

Purpose: Compute the CAVI update for a single survival coefficient β_k via scalar EBNM.

Implementation steps:

1. Compute precision: $A_k = \max(\text{sum}(w * EL2_k), A_floor) \rightarrow A_k$
2. Compute signal: $B_k = \text{sum}(w * z_no_k * EL_k) \rightarrow B_k$
3. Form pseudo-observation: $x_k = B_k / A_k \rightarrow x_k$
4. Form noise level: $s_k = 1 / \text{sqrt}(A_k) \rightarrow s_k$
5. Call EBNM: `ebnm(x = x_k, s = s_k, prior_family = prior_family)`
6. Extract posterior mean, SD, second moment

Parameters:

Parameter	Type	Default	Description
w	n-vector	—	Cox precision weights
z_no_k	n-vector	—	Partial working response (from <code>compute_z_no_k</code>)
EL_k	n-vector	—	Posterior means of $l_{\{ik\}}$ for factor k
EL2_k	n-vector	—	Posterior second moments $E[l^2_{\{ik\}}]$
prior_family	string	"point_normal"	EBNM prior family
A_floor	scalar	1e-10	Minimum value for A_k (prevents division by zero)

Returns: Named list

Field	Description
mean	Posterior mean $\mathbb{E}_q[\beta_k]$
second	Posterior second moment $\mathbb{E}_q[\beta_k^2] = \text{sd}^2 + \text{mean}^2$
sd	Posterior standard deviation
A	Precision A_k (after floor)
B	Signal B_k
x	EBNM pseudo-observation $x_k = B_k / A_k$
s	EBNM noise $s_k = 1 / \sqrt{A_k}$
ebnm_result	Full object returned by <code>ebnm()</code>

A_floor rationale: When all weights w are zero (e.g., at initialisation or in degenerate cases), A_k would be zero and x_k, s_k would be undefined. The floor 1e-10 prevents NaN/Inf while keeping A_k negligibly small — the EBNM prior then dominates and shrinks β_k to zero, which is the correct behaviour.

s_k **direction:** Note that $s_k = 1/\sqrt{A_k}$ is **smaller** when A_k is larger (more data precision $\rightarrow$ tighter pseudo-likelihood). This is the standard EBNM parameterisation: smaller s means the pseudo-observation is more reliable.

3.3 update_beta_all

```
update_beta_all(w, z, EL, EL2, EBeta, prior_family = "point_normal", A_floor = 1e-10)
```

Purpose: Update all K survival coefficients using a Gauss-Seidel sweep.

Implementation: For $k = 1, \dots, K$: 1. Compute z_no_k using the **current** EBeta (already updated for $k' < k$ in this sweep) 2. Call `update_beta_k` 3. Immediately write the new posterior mean into `EBeta_curr[k]`

The Gauss-Seidel pattern (update-and-use within the loop, rather than batch update at the end) is one of the V2 improvements over V1. It accelerates convergence by propagating each β_k update immediately to subsequent factors within the same sweep.

Parameters:

Parameter	Type	Description
<code>w</code>	n-vector	Cox precision weights
<code>z</code>	n-vector	Full working response
<code>EL</code>	$n \times K$ matrix	Posterior means of L
<code>EL2</code>	$n \times K$ matrix	Posterior second moments of L
<code>EBeta</code>	K -vector	Current posterior means of (initialisation for sweep)
<code>prior_family</code>	string	EBNM prior family (passed through)
<code>A_floor</code>	scalar	Precision floor (passed through)

Returns: Named list

Field	Description
<code>EBeta</code>	K -vector of updated posterior means
<code>EBeta2</code>	K -vector of updated posterior second moments
<code>details</code>	Length- K list; each element is the return value of <code>update_beta_k</code> for that factor

4 Test Suite

24 tests across 9 groups. Run with:

```
Rscript tests/run_tests.R
```

All 24 tests pass as part of the 105-test suite.

4.1 T1: Mathematical Identity Checks (6 tests)

Verifies that the raw computations match the mathematical formulas exactly.

Test	What is checked
T1.1	$A_k = \text{sum}(w * EL2_k)$ with floor applied correctly
T1.2	$B_k = \text{sum}(w * z_no_k * EL_k)$
T1.3	$x_k = B_k / A_k$
T1.4	$s_k = 1 / \sqrt{A_k}$
T1.5	second moment = $sd^2 + \text{mean}^2$ (law of total variance identity)
T1.6	All returned values are finite (no NaN, no Inf)

4.2 T2: Non-Informative Prior / WLS Limit (2 tests)

When $EL2_k = EL_k^2$ (zero posterior variance in L), the EBNM problem reduces to a standard weighted least squares problem, and x_k equals the WLS estimator. With a large signal-to-noise ratio, the posterior mean should be close to x_k (minimal shrinkage).

Test	What is checked
T2.1	$EL2 = EL^2 \rightarrow x_k$ equals WLS estimator
T2.2	Large SNR $\rightarrow$ posterior mean x_k

4.3 T3: Known Signal Recovery K=1 (2 tests)

Simulates Cox survival data with a known true and checks that the update recovers it.

Test	What is checked
T3.1	Recover positive = 2.0 from simulated data
T3.2	Recover negative = -1.5 from simulated data

4.4 T4: Multi-Factor K=5 (3 tests)

Exercises update_beta_all with K=5 factors.

Test	What is checked
T4.1	Output vectors EBeta and EBeta2 have length K
T4.2	All second moments non-negative
T4.3	Signs of recovered match true values (2, -1.5, 0) with strong signal

4.5 T5: Null Factor Shrinkage (2 tests)

The point-normal prior should shrink noise factors toward zero.

Test	What is checked
T5.1	Pure noise $z_{no_k} \rightarrow \beta_k$ shrunk toward zero
T5.2	$EL_k = 0 \rightarrow B_k = 0 \rightarrow x_k = 0 \rightarrow \text{posterior mean} = 0$

4.6 T6: Error-in-Variables (1 test)

Higher EL2 (more uncertainty about L) leads to higher A_k , which leads to smaller x_k and more shrinkage. Also confirms that s_k is smaller when A_k is larger.

Test	What is checked
T6.1	$EL2_high > EL2_low \rightarrow A_high > A_low \rightarrow x_high < x_low $; and $s_high < s_low$

4.7 T7: Numerical Stability (5 tests)

Test	What is checked
T7.1	All-zero weights $\rightarrow$ A hits floor, no NaN/Inf returned
T7.2	Extreme $w = 1e8 \rightarrow$ no overflow
T7.3	$n = 1$ edge case (single observation) works correctly
T7.4	Custom A_floor value is respected
T7.5	Variance non-negative: second mean ² always holds

4.8 T8: Gauss-Seidel Behaviour (2 tests)

Test	What is checked
T8.1	compute_z_no_k correctly excludes factor k (z_no_k + EL[,k]*EBeta[k] reconstructs z - z_{-k})
T8.2	Within-loop updates propagate: $\bar{\beta}_1$ updated before $\bar{\beta}_2$ is computed

4.9 T9: V2.R Consistency (1 test)

Test	What is checked
T9.1	update_beta_k output matches lines 356–361 of Supervised_Bayesian_MF_V2.R to within 1e-12

5 Demos

5 narrative scenarios illustrating the update in isolation. No PASS/FAIL output — pure demonstration.

5.1 Demo 1: Anatomy of update_beta_k()

Sets up a single-factor problem with true $\mu = 1.5$, constructs w , z_no_k , EL_k , $EL2_k$ by hand, and calls `update_beta_k`. Prints A_k , B_k , x_k , s_k , posterior mean, and posterior SD. Purpose: show every intermediate quantity and how they connect.

5.2 Demo 2: Signal Recovery K=1

Generates $z_no_k = \mu * EL_k + \text{noise}$ for two scenarios: $\mu = 1.5$ and $\mu = -1.2$. Shows that the posterior mean tracks the true μ closely when the signal is strong relative to the noise.

5.3 Demo 3: Null Factor Shrinkage

Compares two z_no_k vectors: one with real signal (z_no_k correlated with EL_k) and one that is pure noise. Demonstrates that the point-normal prior shrinks the null factor's posterior mean to approximately zero while leaving the signal factor's mean near its true value.

5.4 Demo 4: Error-in-Variables Effect

Holds the signal fixed and varies the second moment $EL2_k$ between a low-uncertainty case ($EL2 \ll EL^2$) and a high-uncertainty case ($EL2 \gg EL^2$). Shows that higher L uncertainty $\rightarrow$ higher $A_k \rightarrow$ smaller $|x_k| \rightarrow$ more posterior shrinkage. Confirms the direction of s_k as a function of A_k .

5.5 Demo 5: Multi-Factor K=5

Constructs a K=5 problem with true $\beta = (1.5, -1.2, 0.8, -0.5, 0.0)$ and calls `update_beta_all`. Loops over the returned details list, printing the posterior mean for each factor and confirming sign recovery. Factor 5 (true $\beta = 0$) is shown to be shrunk near zero by the point-normal prior.

6 Design Decisions and V2 Improvements

6.1 Gauss-Seidel vs. Jacobi Updates

V1 used Jacobi-style (batch) updates: compute all new β_k values using the old EBeta, then replace. V2 uses Gauss-Seidel: immediately write each updated β_k into EBeta_curr before computing z_no_k for the next factor. Gauss-Seidel is generally faster to converge and is implemented by passing EBeta_curr by reference within the loop.

6.2 Precision Floor

The `A_floor = 1e-10` default prevents numerical failure in degenerate cases (all-zero weights, all-zero loadings). It is exposed as a parameter so callers can tighten or loosen it for specific use cases.

6.3 Separation from L and F Updates

β_k appears only in the Cox survival likelihood, not in the genomics reconstruction. This means: - The β_k update does not depend on σ^2 (the genomics noise precision) - F does not enter the β_k update at all - Only L, w, and z are needed

This clean separation makes `update_beta.R` entirely self-contained given w, z, EL, and EL2.

7 Common Pitfalls

Pitfall	Correct behaviour
Using EL^2 instead of EL for A_k	$EL = \text{Var}_q(l) + EL^2$ — always use the full second moment
Assuming s_k increases with A_k Batch (Jacobi) updates	$s_k = 1/\sqrt{A_k}$ — s_k decreases as A_k increases Use Gauss-Seidel: update $EBeta_curr[k]$ immediately after each factor
Forgetting A_floor	Without the floor, zero-weight cases produce NaN from 0/0
Treating EBNM as vector	update is scalar EBNM (single x_k , single s_k) — unlike L and F updates

8 Related Files

File	Role
code/update_beta.R	This module
code/Supervised_Bayesian_MF_V2.R	Main algorithm; lines 356–361 are the inline update matched by T9.1
code/update_L.R	q(L) update; shares compute_z_no_k concept (dual-source EBNM)
code/update_F.R	q(F) update; pure genomics, no involvement
code/update_tau.R	q() update; independent of
code/SupervisedMF_Context.md	AI/code quick-reference for the full implementation
tests/test_update_beta.R	24-test suite for this module
tests/test_helpers.R	Lightweight assertion framework used by tests
tests/run_tests.R	Master test runner (105 tests total)
demos/demo_update_beta.R	5 narrative demonstration scenarios
derivations/qB/qBeta_update_derivation.tex	Self-contained 11-page derivation of q()
derivations/qB/qBeta_update_derivation.pdf	Compiled PDF of the derivation
derivations/MF_UpdateDerivations/MF_V2_CompanionCode.mpp	Mathoncode mapping (17 pages)